

a) Quick Sort vs Merge Sort for Linked list of 10 million

Quick Sort	Merge Sort
Average time ($O(n \log n)$)	Average time ($O(n \log n)$)
Worst case time ($O(n^2)$)	Worst case ($O(n \log n)$)
Extra space ($O(\log n)$) average recursion	$O(\log n)$ for linked list recursion
Can be in place	Can be in place for LL
Poor Cache behaviour	Better Sequential access
Difficult Linked list splitting	Easy splitting
Usually not stable	Stable

Linked list: Merge sort is preferable because splitting and merging can be done efficiently with pointers. $O(n \log n)$ time

Arrays: Quick sort is preferred because arrays allow random access and Quick sort has good cache performance and low extra space $O(n \log n)$ time

b) Heap sort vs Quick sort for 1 Billion Integers

Heap Sort	Quick Sort
Best Case $O(n \log n)$	$O(n \log n)$
Average Case $O(n \log n)$	$O(n \log n)$
Worst Case $O(n \log n)$	$O(n^2)$
Extra Space $O(1)$	$O(\log n)$ average
High Predictability	low
Guaranteed $O(n \log n)$	No worst case guarantee

Analysis:

For 1 billion integers, with strict latency requirements, Heap sort is preferable because it guarantees $O(n \log n)$ worst case time and $O(1)$ extra space.

Quick sort may be faster on average, but its $O(n^2)$ worst case makes it less predictable. However, neither algorithm alone can guarantee completion within 1 second without knowing hardware and implementation.